

Lecture 4

Search Algorithms 3

Adversarial Search

Intelligent Systems
Vrije Universiteit

1

Today we will be looking at agents and how they interact with their environments, as well as playing games using adversarial search methods.

Overview

- 1 State Spaces with Costs
- 2 Search with Partial Information
- 3 Games
- 4 Minimax
- 5 Alpha-Beta Pruning
- 6 Playing Schnapsen

Part 1: State Spaces with Costs

Intelligent Systems
Vrije Universiteit

3

In an ideal environment, the agent can observe everything. But what do we do when we have partial (imperfect) information?

A Search

A **search** is the most well known form of best-first search which evaluates nodes by combining two costs

$g(n)$, the cost to reach the node from initial node

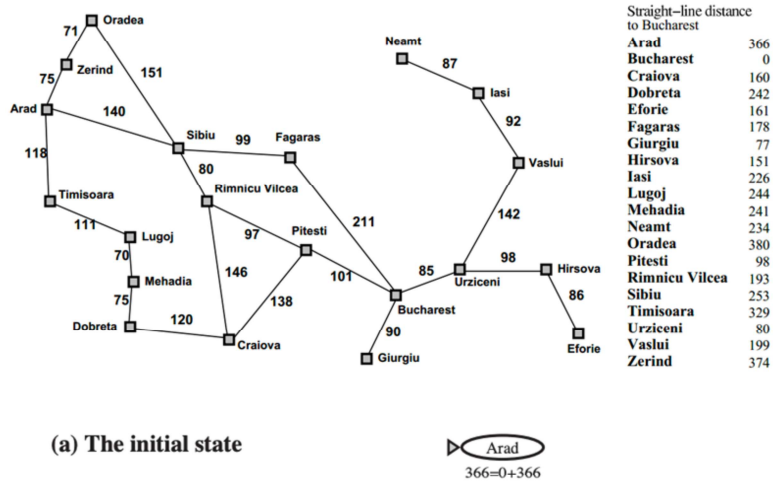
$h(n)$, the cost from node to goal from current node (heuristic)

$$f(n) = g(n) + h(n)$$

where $f(n)$, estimated total cost of path through n to the goal.

In addition to the heuristic value of the successor node $h(n)$, **A star** evaluates the cost that has already been made to reach that node from the initial node $g(n)$. These two functions are summed to result in the evaluation function $f(n)$, which represents the costs made so far and the estimate for what is still needed to solve the problem.

A Search Example



$$f(n) = g(n) + h(n)$$

Arad

$g(n) = 0$, the cost so far is zero.

$h(n) = 366$, SLD to Bucharest in km.

$f(n) = 0 + 366$, estimated cost of path to goal.

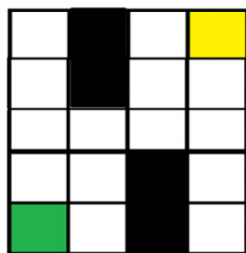
The function $f(n)$ is calculated for every node along the search path, including the initial state Arad. $g(n)$, or the cost so far, is zero because we have not traveled yet. $f(n)$, or the distance to the goal, is 366, derived from the straight-line-distance to Bucharest on the right.

Thus, $f(n) = 0 + 366 = 366$.

Practice Problem

Apply each of the following algorithms to the following problem by creating a search tree. Think of different heuristics or way to measure distance.

Problem find shortest path from tile (1,1) to (4,5) with actions up, down, left, or right.

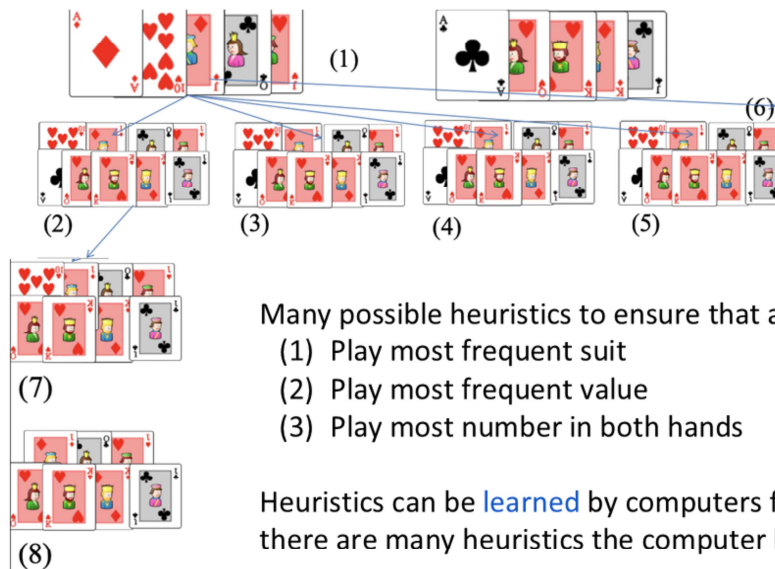


Algorithms

BFS
DFS
DFIS
Greedy BFIS
Hill Climbing
A/A*

Use this problem to practice applying your knowledge of the search algorithms which we have covered so far. Generate a search tree that can be searched to reach tile (4,5) from (1,1). For the informed search algorithms that require using a heuristic, try applying common heuristics we have covered, such as straight-line-distance and Manhattan distance.

Heuristic Functions Schnaptience



Recall [Schnaptience](#), a single-player game where you must alternate playing a card from each deck which either (1) matches suit or (2) matches value.

Many possible heuristics to ensure that all cards can be played eventually:

- (1) Play most frequent suit
- (2) Play most frequent value
- (3) Play most number in both hands

Heuristics can be [learned](#) by computers for games. For example in chess, there are many heuristics the computer has learned and can alternate using.

Heuristics are usually very task-specific, but they all share a common goal; finding the best path to reach the goal state. In the case of Schnaptience, the goal is to play all cards in both hands of the player such that each subsequent card either matches suit or value or the previous one. Heuristics like (1) playing the most frequent suit or (2) playing the most frequent value that the player is holding is a more intelligent method of choosing which card to play next. But there are also heuristics that could be implemented that do not increase performance of the player, such as always playing the lowest value cards.

These kinds of heuristics are what motivate the actions of NCPs in computer games such as chess.

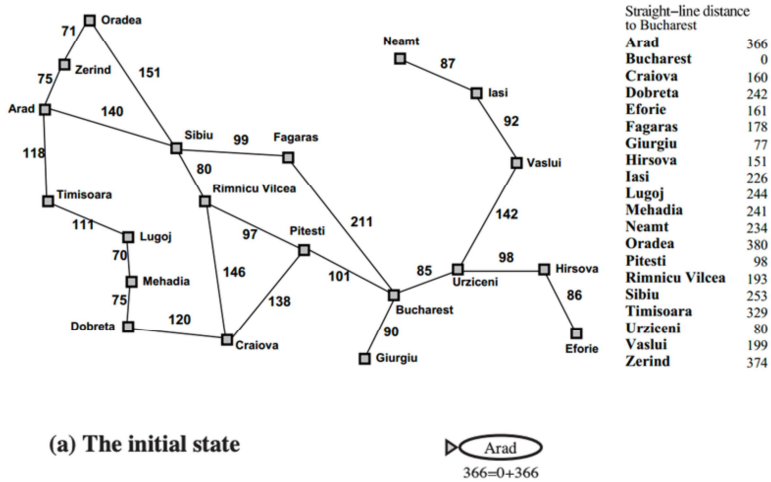
Part 2: Exploration vs Planning and Search with Partial Information

Intelligent Systems
Vrije Universiteit

8

In an ideal environment, the agent can observe everything. But what do we do when we have partial (imperfect) information?

A Search Example



$$f(n) = g(n) + h(n)$$

Arad

$g(n) = 0$, the cost so far is zero.

$h(n) = 366$, SLD to Bucharest in km.

$f(n) = 0 + 366$, estimated cost of path to goal.

The function $f(n)$ is calculated for every node along the search path, including the initial state Arad. $g(n)$, or the cost so far, is zero because we have not traveled yet. $f(n)$, or the distance to the goal, is 366, derived from the straight-line-distance to Bucharest on the right.

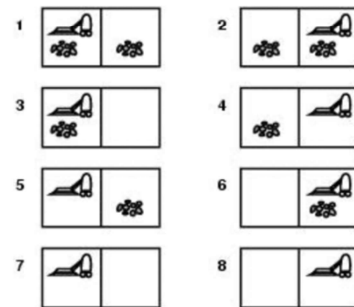
Thus, $f(n) = 0 + 366 = 366$.

Search with Partial Information

If an agent only has **partial information** the environment is not fully observable or deterministic.

This is the case for **sensorless problems**;

- No knowledge of current state
- Agent may perform unnecessary actions



Vacuum World

Partial information means that an agent does not have access to all relevant knowledge for a problem. It may either partially observable or stochastic. In **sensorless problems** the agent does not have any sensors with which to gain knowledge about its environment, including its current state.

This is the case for Vacuum World, in which the vacuum does not have sensors that indicate whether the current room is clean or not clean. As a result, the vacuum may perform unnecessary actions, such as cleaning a room which is already clean.

Search with Partial Information

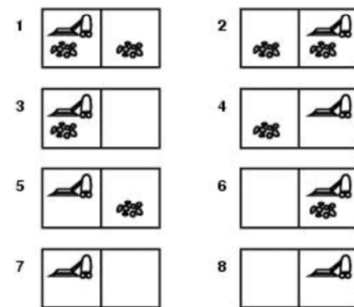
Solution? Let agents have **belief states**, the agent's *current belief* about possible *physical states* it may be in.

Problem

Vacuum starts in any physical state 1-8

Solution

{Right, Suck, Left, Suck}



Vacuum World

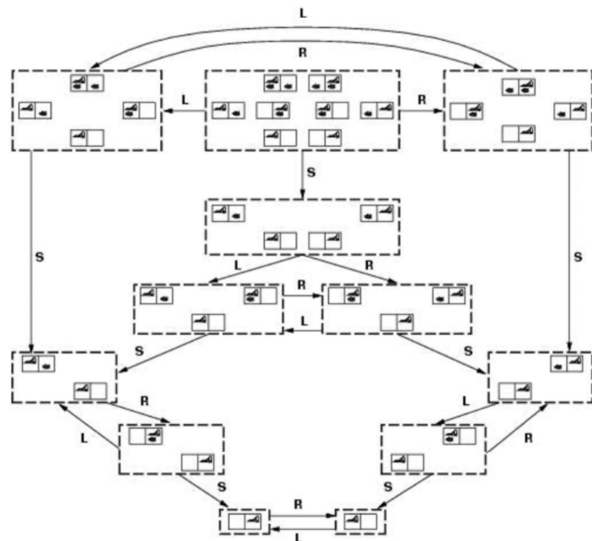
Rather than have awareness of physical states using sensors, the agent uses **belief states** to reach the goal state. For example, regardless of which state 1-8 the agent starts in, if we move to the right, the agent knows it must be in state 2, 4, 6, or 8.

In this Vacuum World example, the vacuum may be in any of the eight physical states and may either be clean or dirty. The vacuum's belief state dictates that there are two rooms which may be clean or empty. Thus, regardless of the actual state, if the vacuum goes to the right room (or stays there), sucks, then moves to the left room and sucks, both rooms will be clean.

Belief State of Vacuum World

Solution belief state where all member states are goal states.

s physical states = 2^s belief states



12

The full belief state of Vacuum World is shown on the right. For s physical states in vacuum world there will be 2^s total belief states. A belief state is a subset of the Powerset of states. Recall that the Powerset is the set that contains every possible *subset*, or combination of elements.

Reasoning using belief states is not an efficient strategy, because the agent may perform unnecessary actions in order to guarantee that it reaches the goal state. However, sensorless problem may be useful in the real world because not having sensors means cheaper manufacturing for a product.

Part 3: Games

Intelligent Systems
Vrije Universiteit

13

When considering search methods for multiagent settings, we can easily model games as a type of adversarial environment.

Classifying Games

	Deterministic	Stochastic (chance involved)
Perfect Information	Chess Checkers Schnapsen Phase 2	Backgammon Monopoly
Imperfect Information	Schnapsen Phase 1	Bridge Poker Scrabble

14

We can define the environment of a game in terms of how much *information* we have and how *predictable* it is. We will look more into these environment types in the next section.

Deterministic means that a game is fully dependent on the actions of each player, while **stochastic** means there is some uncontrollable element, for example rolling of dice. If we have perfect information, this means that the agent can see all relevant factors of the current state to the decision or action it has to take. Schnapsen Phase 1 is an example of imperfect information because the agent can not see all the cards in the deck.

Games

Games are a form of competitive, multi-agent environments. Agents try to win over their **adversary**, giving rise to adversarial problems:

- What are other agents doing? How do their actions affect my success?

Why study games?

- Historically fun and entertaining
- Interesting subject of study because they are difficult
- Easy to represent
- Agents restricted in terms of possible actions

15

Adversarial search is a type of search we use when we are dealing with more than one agent. This type of search is useful to multiplayer games in which two or more adversaries are trying to maximise their chances of reaching a desired outcome.

We care about games because they are present in almost every culture and are easy to represent digitally because they are governed by rules.

Relating Games to Search Problems

Search

No adversary

Solution = heuristics to reach goal

Heuristics can find the *optimal* solution

Actions taken are defined by optimal solution

Games

Adversary

Solution = strategy for every new move

Time limit forces *approximate* solution

Evaluation function for quality of current game position

16

We can use adversarial search methods to play games effectively, but there are key differences in how we define **search** itself vs. **games**. While the search methods we have looked at so far (informed, uninformed) had no adversary, games involve other players that change the possible states *during* the game. Thus, the possible solutions in every new state changes and we need a strategy to react to the opponent's moves.

In some applications of (un)informed search, we can find an optimal solution, but time limits present for most games prevent us from doing an endless search for the perfect move. As a result, our evaluation function finds the best move within a reasonable time in order to win.

How do we treat games as search?

Initial state: starting board configuration

e.g. empty tic-tac-toe board

Successor function: pairs of legal moves (*move, state*)

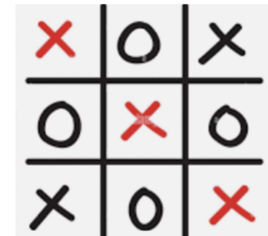
e.g. empty boxes for X or O

Terminal test: is the game finished?

e.g. does a player have 3 in a row or are all boxes full?

Utility function: gives numerical value of terminal states

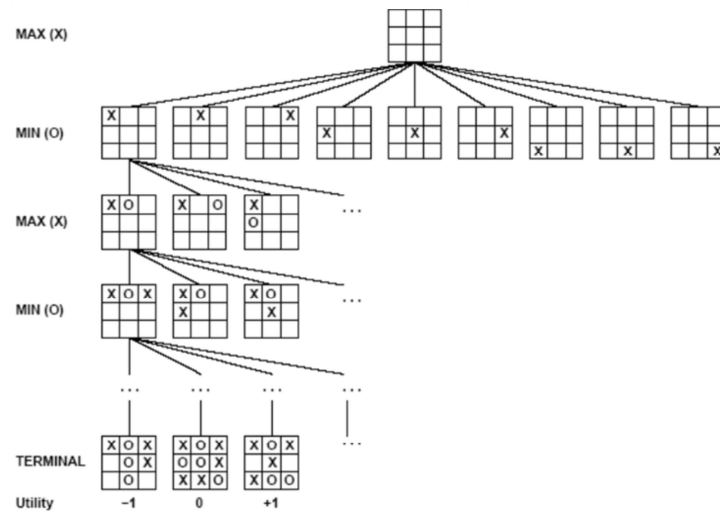
e.g. Win (+1), Lose (-1), Draw (0)



X	O	X
O	X	O
X	O	X

We can create analogies between search and the necessary components for rationally playing games. The **initial state**, which we are familiar with already, is analogous to the starting position of a game, for example an empty tic-tac-toe board. **Successor functions** in the case of tic-tac-toe are the slots still available and the X or O player making a marking. **Terminal test** for tic-tac-toe has two conditions: does one player have three markings in a row or are all the boxes full? The **utility function** is a function of either of these terminal states, which is +1 for the winning agent, 0 if draw, or -1 for the losing agent.

Partial Game Tree for Tic-Tac-Toe



This is a partial game tree depicting minimax with MIN as the O player and MAX as the X player for a game of tic-tac-toe.

Part 4: Minimax

Intelligent Systems
Vrije Universiteit

19

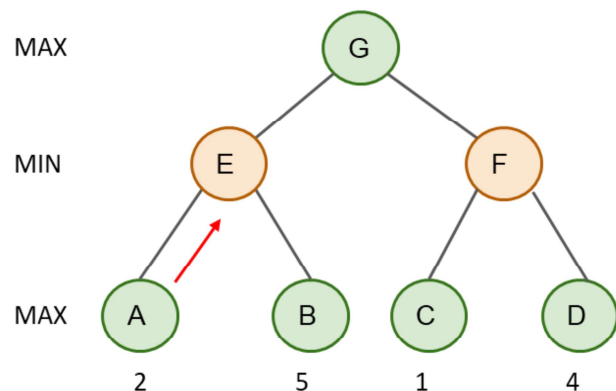
This section will introduce our first adversarial search method: Minimax.

Minimax

Minimax is an adversarial search method used for games that uses two players called *max* and *min*.

- Each layer of the search tree is either min or max's turn

Method: start with the leaf node values and backtrack up through the tree to find the optimal move for each player.



20

The minimax algorithm is a backtracking algorithm that starts at the bottom of a search tree and works upwards. The value that is chosen to continue up to the next node is decided by whose turn it is: either MAX player or the MIN player.

For example, in the tree shown the MIN player has the first turn. if node A has value 2 and node B has value 5, the MIN player will choose to take the lower value of 2 and make this the new value of node E.

Minimax Strategy

MAX tries to *maximise* the value. MIN tries to *minimise* the value. The assumption is that both players play optimally.

- If MIN or MAX doesn't play optimally, then the other player will perform even better

Given a game tree, optimal strategy can be determined using minimax value of each node

MINIMAX-VALUE(n)=

UTILITY(n)

If n is a terminal

$\max_{s \in \text{successors}(n)} \text{MINIMAX-VALUE}(s)$

If n is a max node

$\min_{s \in \text{successors}(n)} \text{MINIMAX-VALUE}(s)$

If n is a min node

21

The minimax algorithm works under the assumption that both players are playing optimally, meaning they are choosing the values in their best interest. The optimal strategy is decided by the value of the nodes possible according to the successor function.

Minimax Algorithm

Algorithm 1: Recursive Minimax

Data: Starting state node S

Function Minimax(S , $Maximizing = True$):

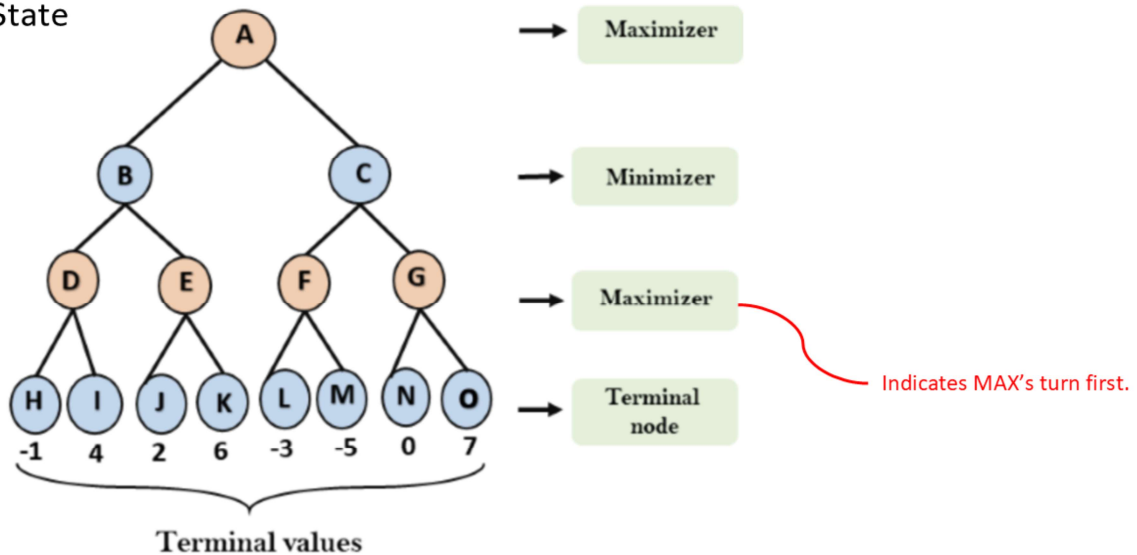
if S is terminal then	}	IF terminal node, return its value
return Utility(S)		
if $Maximizing = True$ then	}	IF MAX's turn, pick node with highest value
$v \leftarrow -\infty$;		
for each child in S do		
$v \leftarrow \text{Max}(v, \text{Minimax}(\text{child}, False))$;	}	
end		
return v ;		
else	}	ELSE (MIN's turn), pick node with lowest value
$v \leftarrow +\infty$;		
for each child in S do		
$v \leftarrow \text{Min}(v, \text{Minimax}(\text{child}, True))$;	}	
end		
return v ;		
end		

22

The minimax algorithm presented here can be interpreted in 3 parts: if the current node is a terminal node, then return its value. Otherwise, choose the node which maximises/minimises the expected value depending on which player's turn it is.

Minimax Example

Initial State

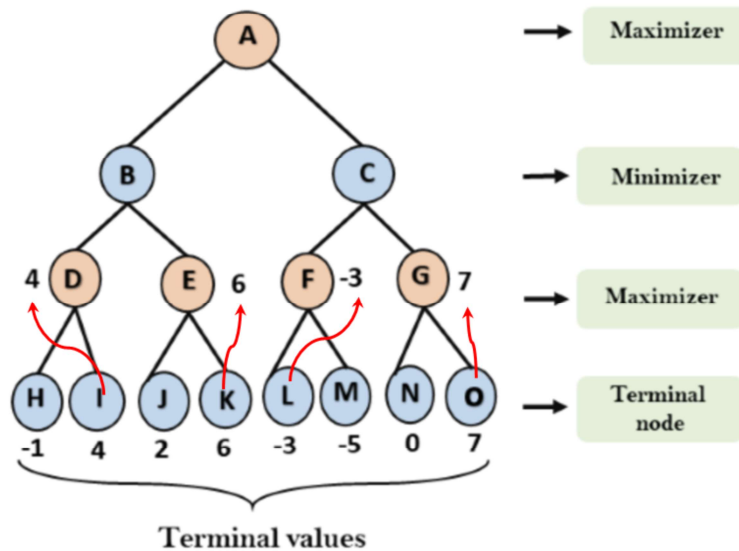


23

Let's work out our first full minimax problem.

Starting from the leaf nodes, we see that MAX has the first turn. MAX can choose the values that will be given to nodes D, E, F, and G.

Minimax Example



For each parent node, MAX chooses the child with the *highest* value in order to maximise the utility value.

These values are adopted by the parent.
e.g. Node D
 $\max(-1, 4) = 4$

24

MAX will pick the highest out of all possible values for each node in this layer and these values are carried up.

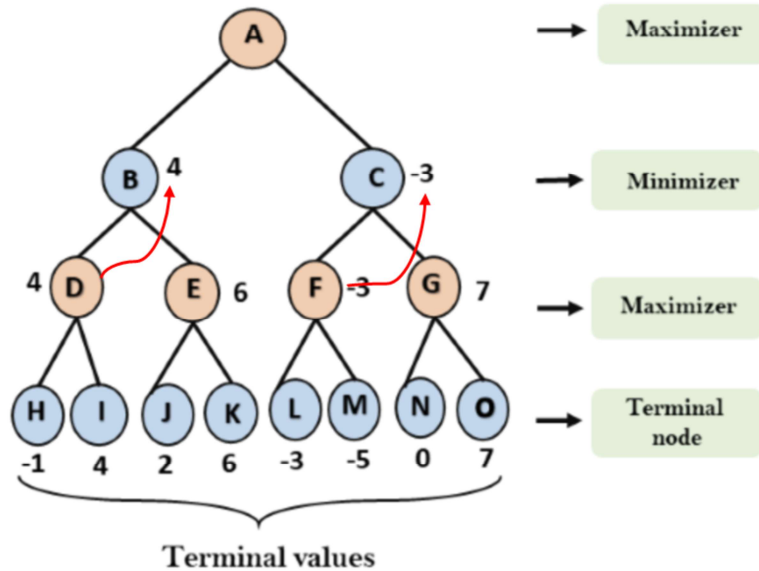
Node **D**: $4 > -1 \rightarrow 4$

Node **E**: $6 > 2 \rightarrow 6$

Node **F**: $-3 > -5 \rightarrow -3$

Node **G**: $7 > 0 \rightarrow 7$

Minimax Example



For each parent node, MIN chooses the child with the *lowest* value in order to minimise the utility value.

These values are adopted by the parent.
e.g. Node B
 $\min(4, 6) = 4$

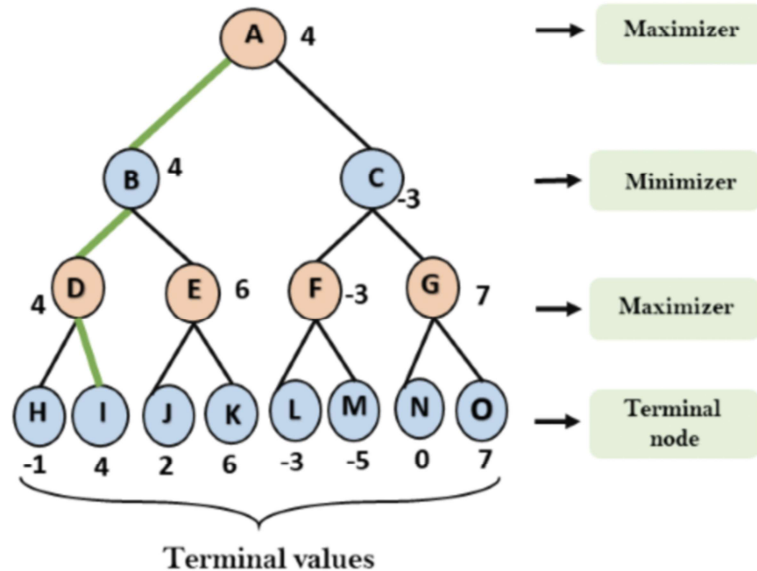
25

MIN will pick the lowest out of all possible values for each node in this layer and these values are carried up.

Node **B**: $4 < 6 \rightarrow 4$

Node **C**: $-3 < 7 \rightarrow -3$

Minimax Example



MAX has the final turn and chooses the highest out of all possible children.

MAX will pick the highest out of all possible values for each node in this layer and these values are carried up.

Node **A**: $4 > -3 \rightarrow 4$

4 is the final value of the root node.

Minimax Performance

Performance Measure	Minimax
Completeness	YES
Time	b^m
Space	bm
Optimality	YES



Minimax is a complete algorithm because it explores every node in the game tree before making a decision. It is also optimal, because it assumes that MAX and MIN both choose the value which optimises their expected value. However, time complexity is very poor because it grows exponentially with the maximum depth of the tree.

For games like tic-tac-toe, minimax can be very effective because of the limited number of possible moves and finite number of total moves. However, for games like chess, the time and space complexity makes it too computationally expensive.

Utility vs. Heuristics

Utility means that value is based on quality of the state, e.g.

- Player X gets 33 points, player Y gets 68
- Player X wins with 3 points

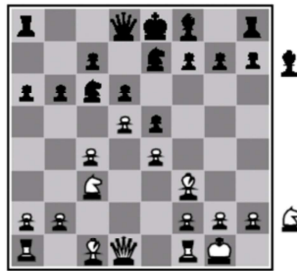
Heuristic Evaluation means value is based on **estimation** of the quality of the state. In a board game, this function calculates the value of the board depending on piece placement.

- Evaluation function changes per game
- 2 pawns and a bishop is stronger than a castle

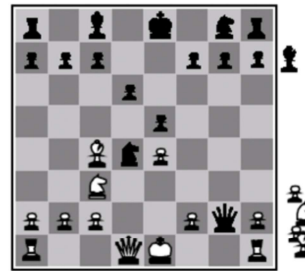
Utility and **heuristic evaluation** are both methods of assessing how good a particular state is at any point. The key difference is that heuristic evaluation makes an estimate of the quality of a state. For example, it takes into account how much closer a particular state is to the goal state of the game. By contrast, the utility is more simply the numerical evaluation of a state.

EVAL Heuristics *Example*

For chess, the evaluation function is usually linear.



Black to move
White slightly better



White to move
Black winning

$$Eval(s) = w_1f_1(s) + w_2f_2(s) + \dots + w_nf_n(s)$$

The evaluation function used for a chess application is often linear. It is simple a summation of the estimated value of different moves that the player can make.

From Perfect to Imperfect Information

Minimax requires many leaf-node evaluations, which is impractical if search is to take a reasonable amount of time. Idea:

Sacrifice perfect information for performance.

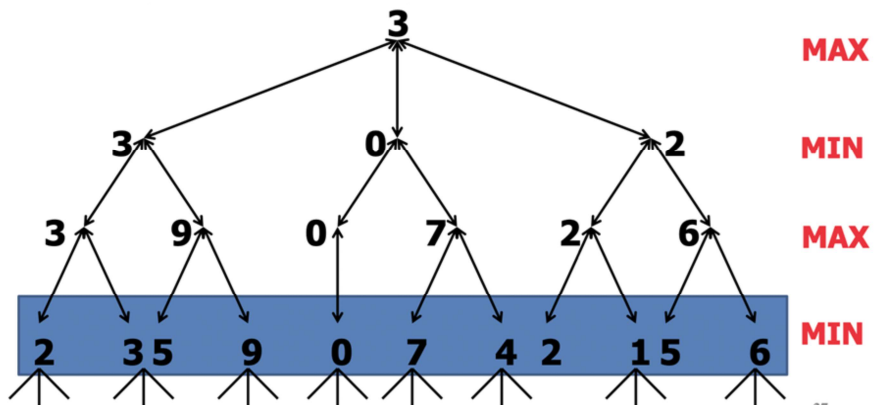
Shannon, 1950

Note: this is the opposite of what happens in Phase 1 Schnapsen, which is turn imperfect information into perfect, and sample over possible options.

30

The minimax algorithm as we've seen thus far works on **perfect information**, meaning that the agent looks at all possible moves with corresponding game states and their value. But we've also seen that this is very computationally expensive. What if we sacrifice some of this information, and thus our performance, in order to have a more efficient search?

Restricted Search Depth



One way to sacrifice perfect information for performance is to restrict the depth of our search. For example, rather than explore every possible path down to the terminal nodes, we stop evaluating at a restricted depth.

Horizon Effect

If our evaluation function has restricted depth search, it may **not see crucial game plays** that occur deeper in the tree.

- E.g. a piece is captured in chess

This is an example of sacrificing performance for information.

The function's **limited horizon** prevents it from seeing this game possibility

The type of restricted depth minimax we just saw will result in the **horizon effect**. Imagine looking at the horizon in a landscape as exploring the depths of a search tree. If you walk towards the horizon, you may see new things that you weren't previously aware of. In terms of restricted depth minimax, not walking forward can result in missing crucial move outcomes that are just beyond the field of view. This is a sacrifice resulting in imperfect information.

Part 5: Alpha-Beta Pruning

Intelligent Systems
Vrije Universiteit

33

Instead of restricting the depth of the minimax algorithm to make it more efficient, we can use another technique called alpha-beta pruning.

Alpha-Beta Pruning

Problem with minimax: # game states is exponential to # of moves!

Alpha-Beta Pruning an optimisation technique we can apply to minimax.

- Don't examine every node in the tree!
- Reduces computation time
- Adds two parameters to our algorithm: **alpha** and **beta**

Alpha: value of best choice found so far on the MAX path

Beta: value of best choice so far on the MIN path

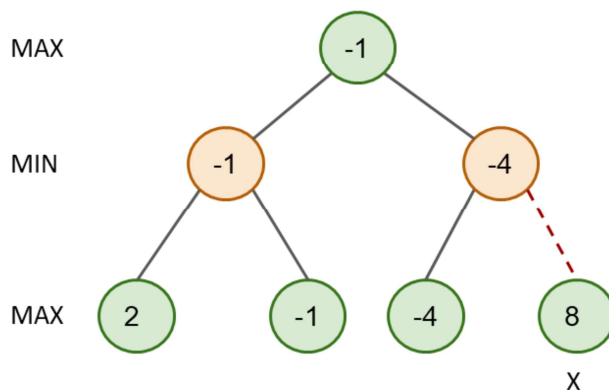
34

As we saw in the last section, the minimax algorithm is complete and optimal, but at the cost of time complexity. Rather than restrict the search depth and suffer from the limited horizon, we can decide which nodes to **prune** (cut) without sacrificing important information.

The **alpha-beta pruning** method adds an alpha and a beta parameter to the minimax algorithm. We can think of alpha as the best choice found by the MAX player, or the *minimum value it will accept*. We can think of beta as the best choice found by the MIN player, or the *maximum value it will accept*.

Alpha-Beta Pruning Example

Sometimes you don't need to explore a node to know it is irrelevant to your search.



We can **prune** the 8! Why?

1. We know that MAX will eventually choose between the top two nodes.
2. We know that it can choose (-1) on the left or (-4) and a mystery node (X) on the right.
3. MIN will only choose X if $X < -4$.
4. Thus, **regardless of the value of X**, we know MAX will choose the node on the left because $-1 > -4$.

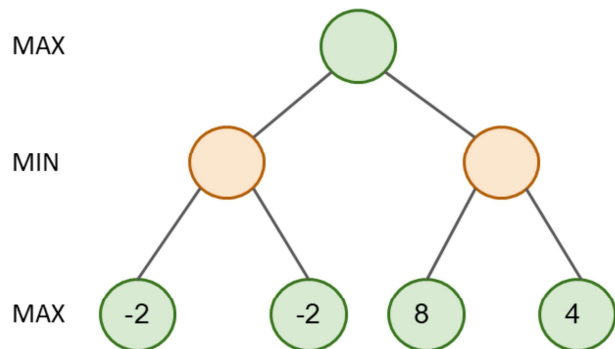
35

Let's look at a solution to an alpha-beta pruning problem before we work one out. Why does the algorithm not need to look at the 8?

We can intuitively reach this answer by thinking about the motive of each player. After we have searched all nodes except the 8, the MAX player can choose between the -1 on the left, the -4 on the right, or maybe the mystery node X (which we now know is 8). The MAX player also knows that, since MIN can choose between -4 and X, it will either choose -4 or a value *even smaller*. Thus, MAX *does not need to see the 8 to know that it will choose the -1 on the left*, because it is bigger than -4.

Alpha-Beta Pruning *Example 2*

Solve using the minimax method, but we update an interval with every move that we make (α , β). The interval starts as $(-\infty, \infty)$.



Let's solve this mathematically rather than intuitively by keeping track of two rules along the way:

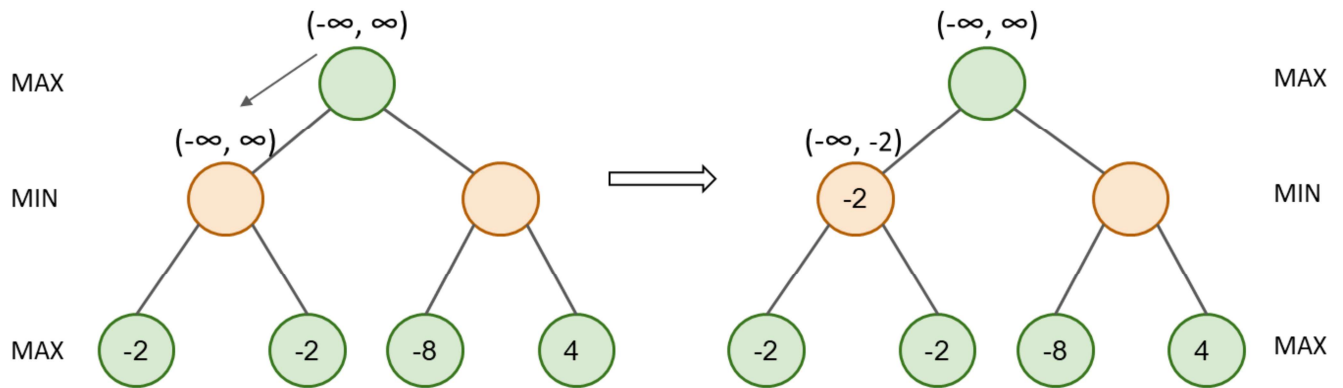
1. MAX can only change α and MIN can only change β
2. When $\alpha > \beta$, prune!

Let's solve an alpha-beta pruning problem from scratch using our parameters α and β . Recall that α is the best choice MAX has, and β is the best choice that MIN has. We can organize them into an interval (α, β) . We need to keep two rules in mind:

- (1) MAX can only change α and MIN can only change β
- (2) When $\alpha > \beta$, prune!

Alpha-Beta Pruning Example 2

1. MAX can only change **alpha** and MIN can only change **beta**
2. When **alpha** > **beta**, prune!



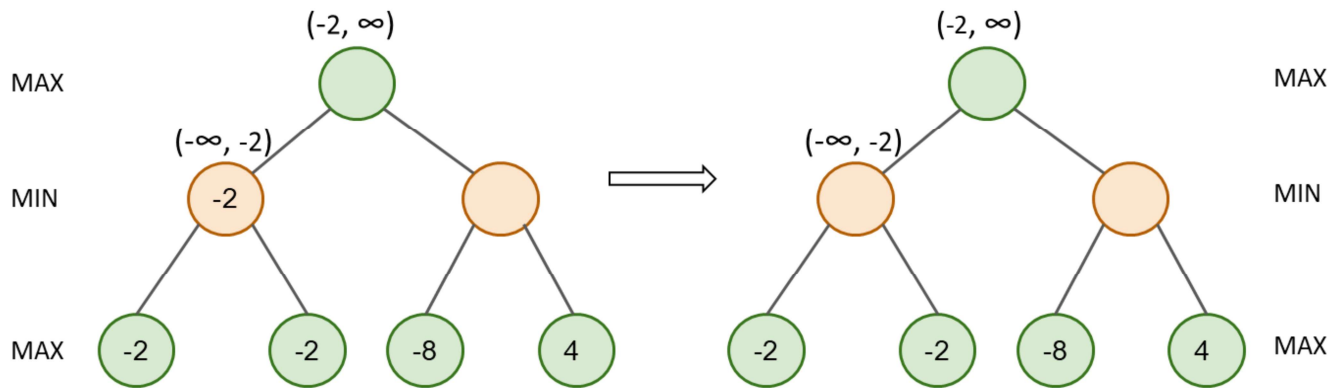
In MIN's first move, it changes beta to -2 and makes the node value -2. Intuitively, MIN is saying "I won't accept a number higher than -2."

37

We always start by assigning the parents node the interval $(-\infty, \infty)$. We carry this interval down the left side. MIN can choose between -2 and -2, and changes the beta value to the node it chooses: $(-\infty, -2)$. This means the best choice it has currently is -2.

Alpha-Beta Pruning Example 2

1. MAX can only change **alpha** and MIN can only change **beta**
2. When **alpha > beta**, prune!



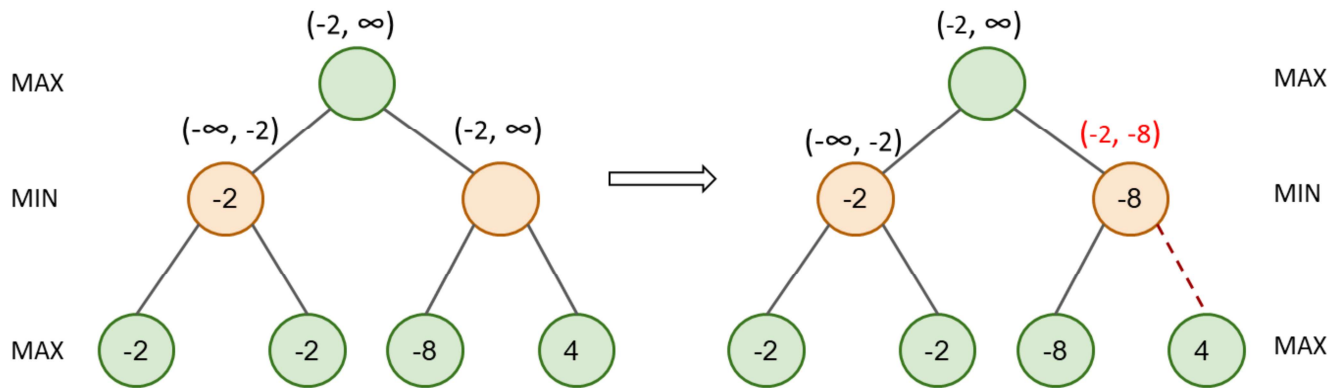
The new interval is **passed up the tree** to the parent node. Since the parent node is MAX, it can only update the alpha value and the new interval becomes $(-2, \infty)$. The interval is **passed down the tree** on the right, where MIN changes the beta value $(\infty, -2)$.

38

The interval $(-\infty, -2)$ is carried *up* to the parent node, but we *flip the values*. Why? Recall that MAX is only allowed to change the alpha value, and the best it can hope for now is -2 or any value greater than that. When this interval is carried down the other side, it does *not* get flipped.

Alpha-Beta Pruning Example 2

1. MAX can only change **alpha** and MIN can only change **beta**
2. When **alpha > beta**, prune!

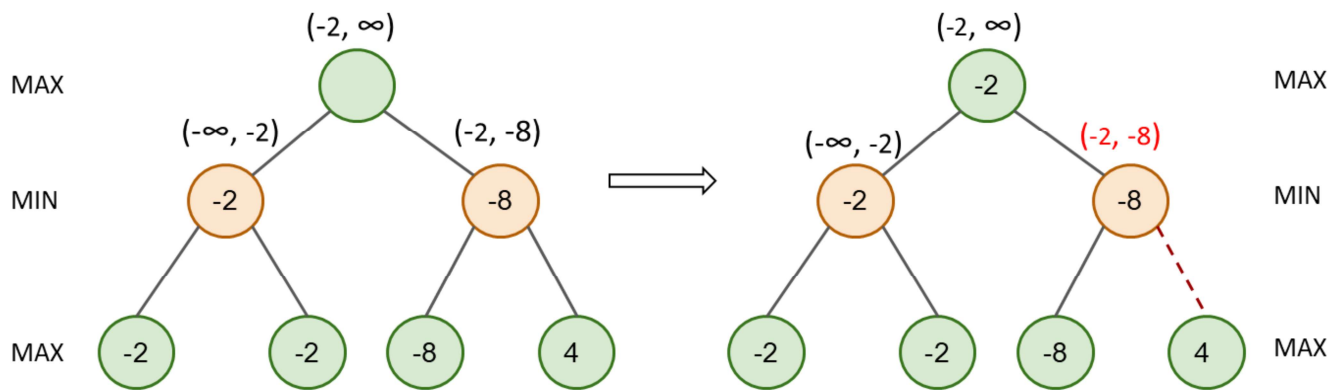


MIN changes alpha to -8 because it's < -2 , making the new interval $(-8, -2)$.
Check rule 2! Since $\alpha > \beta$, we can prune the last node.

Now MIN explores the -8 node and sees that is it *more desirable* than -2 , which is it's current best option. It updates the interval accordingly and changes beta to -8 . Now we can observe that $\alpha > \beta$, thus we prune the node 4 and do not explore it.

Alpha-Beta Pruning Example 2

1. MAX can only change **alpha** and MIN can only change **beta**
2. When **alpha** > **beta**, prune!



The final solution is -2, with one node pruned.

Finally, MAX has the last turn and chooses -2 out of -2 and -8.

Alpha-Beta Algorithm

function ALPHA-BETA-SEARCH(*state*) *returns an action*

inputs: *state*, current state in game

$v \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$

return the *action* in SUCCESSORS(*state*) with value v

function MAX-VALUE(*state*, α , β) *returns a utility value*
if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)

$v \leftarrow -\infty$

for s in SUCCESSORS(*state*) **do**

$v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s, \alpha, \beta))$

if $v \geq \beta$ **then return** v

$\alpha \leftarrow \text{MAX}(\alpha, v)$

return v

function MIN-VALUE(*state*, α , β) *returns a utility value*
if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)

$v \leftarrow +\infty$

for s in SUCCESSORS(*state*) **do**

$v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(s, \alpha, \beta))$

if $v \leq \alpha$ **then return** v

$\beta \leftarrow \text{MIN}(\beta, v)$

return v

The boxed pseudocode demonstrates how each player takes the best option of the values passed up by the other player.

Alpha-Beta Evaluation

- Pruning the tree does *not* affect the final result.
- Entire subtrees can be pruned
- Good **move ordering** improves effectiveness of pruning
- With perfect pruning, time complexity is $b^{m/2}$
 - Alpha-beta pruning can look twice as far as minimax in the same time

42

Applying the alpha-beta pruning technique to minimax does not affect the outcome. This is because nodes are only pruned when their value is irrelevant to the choice that we can predict will be made at the next level.

Move ordering will affect the order in which terminal nodes are lined up. As a result, good move ordering will allow us to prune more effectively, since the algorithm *must* move in a left-to-right fashion through the search tree.

When pruning is done optimally, the time complexity of minimax is halved when alpha-beta pruning is applied. Although it might not sound very impressive, it allows you to double your search depth; in chess you could potentially look 12 moves ahead rather than 6.

Part 6: Playing Schnapsen

Intelligent Systems
Vrije Universiteit

Schnapsen *Phase 1*

In Phase 1, Schnapsen is **deterministic** because there is no element of chance once all cards are arranged.

We have **imperfect information** because there are many possible card deals.

The best strategy: assume perfect information and average

- Play minimax for all possible deals and choose the one most likely to win

$$\operatorname{argmax} \sum P(s) \operatorname{minimax}(\operatorname{result}(s, a))$$

44

Schnapsen Phase 1 is challenged by the fact that it deals with imperfect information. The agent can not see every card in the deck.

The best strategy to win nevertheless is to use the minimax algorithm to search through every possible deal that we can not observe. This way we know for which games we would win or lose. For all possible deals, we then choose the one which is most likely to win. We can do this by averaging over the possible outcomes of each minimax run.

Minimax Phase 1

At the beginning of...

Trick 5: maximum 6 deals

Trick 4: $8 \cdot 7 \cdot 6 = 336$

Trick 3: $10 \cdot 9 \cdot 8 \cdot 7 \cdot 6 = 30,240$

Trick 2: $12 \cdot 11 \cdot 10 \cdot 9 \cdot 8 \cdot 7 \cdot 6 =$
3,991,680

Trick 1 (the game): $14! / 5! =$
726,485,760

Minimax : $5! \cdot 5! \cdot 5^5 \cdot 5^5 = 140,625,000,000$

45

Each player starts with 5 cards in their hand, not visible to the opponent. In trick 5 there are 6 possible deals because there are two cards left in the talon (undealt deck of cards), one of which is the trump card. So there are 6 different possibilities for how the cards are distributed.

When we continue to perform such combinatorics on all the tricks in phase 1, we realize the impossibility of performing minimax on every possible deal.

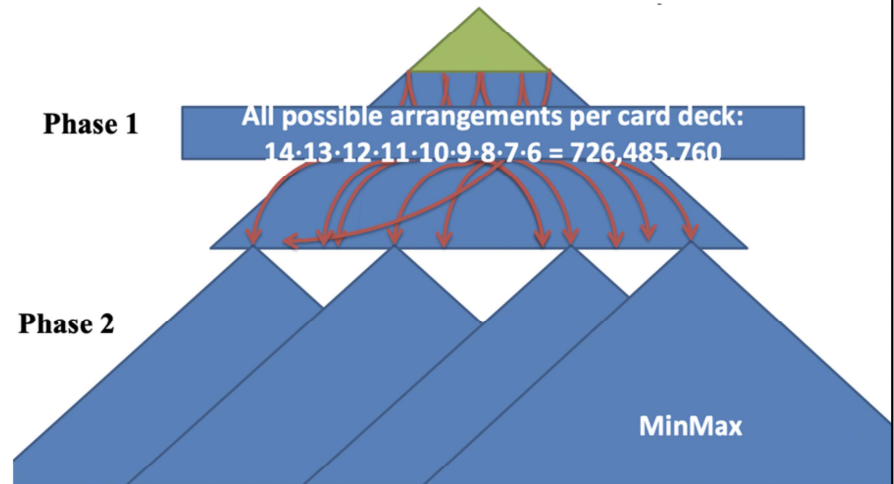
Minimax Phase 2

In total 10^{20} possibilities!

After trick 3 there are 336 possible deals. Minimax will take

$$5! * 5! * 5^2 * 5^2 = 9,000,000$$

$$\text{Complexity} = 3 * 10^9$$



Phase 2 of Schnapsen also results in a number of possible arrangements that is impossible for a computer to calculate with minimax.

Perfect Information Monte Carlo Sampling (PIMS)

PIMS is a sampling technique for playing imperfect information games. It randomly samples n times by playing n games.

$$\operatorname{argmax} (1/n) \sum_{i=1}^n \operatorname{minimax}(\operatorname{result}(s_i, a))$$

Still expensive with 5^{10} nodes to be explored. In the Schnapsen platform we will use, sampling is done with **random rollouts**; play a complete game with one player making random moves and use this to define your weights.

$$\operatorname{argmax} (1/n) \sum_{i=1}^n \operatorname{randomrollouts}(s_i, a)$$

47

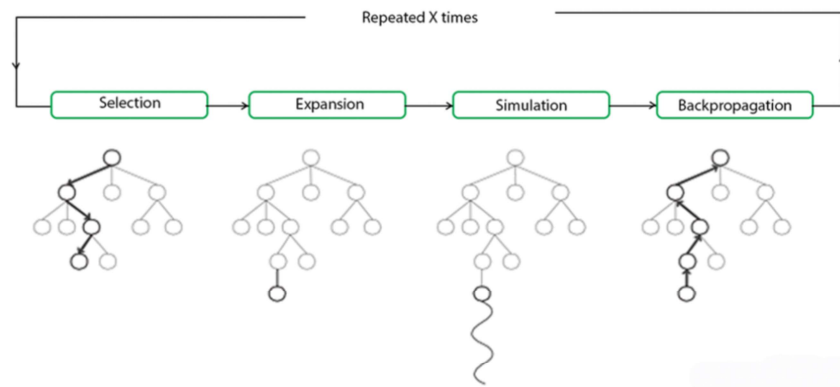
Perfect Information Monte Carlo Sampling can be used to alleviate this computational problem. By sampling, meaning randomly taking a subset, and averaging over this subset, you can generalise the outcome to the total population of subsets. For example, we can choose $n = 1000$ games out of 700 million total possible games to play out using minimax. Although still computationally expensive, we significantly reduce complexity.

The **random rollout** method, used in Schnapsen platforms, plays out a random number n of games and simply checks whether we win or lose in the end.

Monte Carlo Tree Search

Monte Carlo Tree Search is a heuristic search algorithm for deterministic games.

- It is the core of the AlphaGo approach, which learns the structure of the search tree



48

This random rollout approach was also used in Google's AlphaGo (see next slide) with **Monte Carlo tree search** to learn the structure of the search tree.

AlphaGo



AlphaGo vs. Lee Sedol, 2016

49

Go is one of the world's oldest board games originating in East Asia, with more search choices at any given time than atoms in the universe. In 2016, Google DeepMind's AlphaGo beat Lee Sedol, one of the world's top Go players. How? By using a combination of alpha-beta pruning for the minimax algorithm and Monte Carlo tree search.

Intelligent Systems Framework

Efficient, simple, correct, well- understood, generic	Find best decision based on experience.	Weeks 6 & 7
	Find best decision with knowledge & reason.	Weeks 3, 4, 5
	Find best decision in multiple agent setting.	Weeks 1 & 2
	Find best decision with heuristics.	
	Systematically find all possible solutions.	

Progress in the intelligent systems framework.

Summary & Homework

Summary	Homework
Minimax is an adversarial method for two players, min and max. It can be made more efficient using alpha-beta pruning by removing unnecessary nodes. We can use statistical sampling to overcome imperfect information.	Practical assignment 4: Adversarial Search → book slots with TAs Theoretical material in work groups Practice Schnapsen

Thank you for your attention and have a lovely day.